T.C.
MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
MİKROİŞLEMCİLER GİRİŞ DERSİ

Hafıza Kartı Oyunu

PROJE RAPORU

Sümeyye Karaköse - 100921041

İÇİNDEKİLER

Hafıza Kartı Oyunu....0
PROJE RAPORU....0
1. Giriş....2
2. Projenin Amacı ve Kapsamı....2
3. Kullanılan Teknolojiler....2
4. Sistem Tasarımı....3
5. Çalışma Mantığı....3
7. Veri Yapıları ve Değişkenler....4
8. Kullanıcı Arayüzü....5
9. Kodun Yapısı ve Önemli Fonksiyonların/Prosedürlerin Açıklanması....8
10. Karşılaşılan Sorunlar ve Çözümler....13
11. Gelecek Geliştirmeler....14
12. Github Linki....15

1. Giriş

Bu projenin adı Hafıza Kartı Oyunudur. Proje, Assembly dili kullanılarak Emu8086 ortamında geliştirilmiş bir kart eşleştirme oyunudur. Oyunda amaç, kısa süreliğine gösterilen kartların yerlerini hatırlayarak aynı harfe sahip kartları eşleştirmektir.

Bu proje sayesinde kullanıcı ile doğrudan etkileşim kuran, zaman ve hamle gibi değişkenlerle yönetilen, modüllere ayrılmış bir oyun yapısı oluşturulmuştur.

2. Projenin Amacı ve Kapsamı

Bu projenin temel amacı, kullanıcının hafıza ve dikkat becerilerini ölçen bir kart eşleştirme oyunu oluşturmaktır. Oyuncu, kartlar kısa bir süre açık gösterildikten sonra kartların konumlarını hatırlayarak aynı harfleri bulmaya çalışır. Böylece oyun yalnızca bir eğlence aracı değil, aynı zamanda oyuncunun kısa süreli hafıza ve dikkat yeteneğini test eden bir yapı sunar.

Projede kullanıcıdan beklenen şey, önce oyun alanı boyutunu seçmesi, ardından kartları satır ve sütun bilgisi girerek açması ve eşleşmeleri doğru biçimde bulmasıdır. Kullanıcı doğru eşleşmeler yaptıkça kartlar açık kalır; yanlış eşleşmelerde ise kartlar yeniden kapanır ve oyuncu hamle kaybeder. Ayrıca süre sınırı ve hamle sınırı bulunduğu için oyuncunun oyunu dikkatli ve planlı şekilde sürdürmesi gerekir.

Bu proje, özellikle düşük seviyeli programlama ortamında etkileşimli bir oyun üretme ihtiyacını karşılamaktadır. Kart dizileri, ekran çizimi, zamanlayıcı ve giriş kontrolleri Assembly seviyesinde ele alınarak, sistem programlamanın temel mantığı uygulamalı biçimde gösterilmiştir.

3. Kullanılan Teknolojiler

Bu projede temel olarak Assembly dili ve Emu8086 geliştirme ortamı kullanılmıştır. Assembly dili, donanıma çok yakın çalışan, düşük seviyeli bir programlama dilidir. Emu8086 ise bu dili test etmek, çalıştırmak ve simüle etmek için kullanılan bir ortamdır. Projedeki tüm ekran çizimleri, giriş işlemleri ve zaman yönetimi bu ortam üzerinden gerçekleştirilmiştir.

Oyunda hem BIOS interrupt'ları hem de DOS interrupt'ları kullanılmıştır. BIOS kesintileri ekranın konumlandırılması, zaman sayımı ve fare kontrolü gibi işlemlerde görev alırken, DOS kesintileri metin yazdırma ve karakter girişlerinde kullanılmıştır. Örneğin ekran üzerine mesaj yazdırmak için DOS tabanlı yazdırma işlemleri kullanılmış, süre takibi için BIOS zaman kesintisinden yararlanılmıştır.

Ayrıca klavye ve mouse etkileşimi, doğrudan interrupt tabanlı olarak ele alınmıştır. Kullanıcının oyun alanı seçimi, kart satır ve sütun girişi, restart butonuna basılması gibi işlemler bu mekanizmalarla işlenmiştir. Böylece proje, sadece mantıksal bir oyun değil, aynı zamanda ekran, zaman ve giriş yönetimini birlikte kullanan bütünlüklü bir sistem haline gelmiştir.

4. Sistem Tasarımı

Proje, daha düzenli ve anlaşılır bir yapı oluşturmak için birkaç ayrı dosyaya bölünmüştür. Bu yaklaşım sayesinde ekran çizimi, oyun mantığı, kullanıcı girdisi ve yardımcı işlemler birbirinden ayrılmıştır. Assembly gibi düşük seviyeli bir dilde bu tür bir modüler yapı kurmak, kodun hem okunabilirliğini hem de bakım kolaylığını artırmıştır. Kullanılan beş dosya yapısı aşağıda açıklanmıştır:

Ana akış *main.asm* dosyasında yer almaktadır. Bu dosya oyunun başlangıcını, oyun alanı seçimini, ana döngüyü, restart akışını ve oyun sonu koşullarını yönetmektedir. Kullanıcı hangi boyutta oynayacağını bu dosya üzerinden seçer ve oyunun genel geçişleri burada kontrol edilir.

grid.asm dosyası, kartların ekrana çizilmesinden sorumludur. Oyun tahtasının hücreleri bu modülde oluşturulmuş, kartların açık ya da kapalı durumda görünmesi burada yönetilmiştir. Böylece tahta çizimi, oyunun mantığından ayrılmıştır.

logic.asm dosyası, oyunun kurallarını içermektedir. Doğru eşleşme, yanlış eşleşme, hamle kontrolü, kart dizilerinin oluşturulması, kartların karıştırılması ve rastgelelik işlemleri bu modülde gerçekleştirilmiştir. Oyunun temel davranışı bu dosyada şekillenmektedir.

display.asm dosyası, ekran çıktıları ve görsel bileşenler için kullanılmıştır. Başlangıç ekranı, restart butonu, süre göstergesi, hamle bilgisi, kısa önizleme süresi ve uyarı mesajları bu modülde yer almaktadır. Böylece kullanıcıya gösterilen tüm görsel bilgiler tek bir yapıda toplanmıştır.

input.asm dosyası ise kullanıcı girdilerini yönetmektedir. Satır ve sütun seçimi, restart butonuna basılması ve seçim sırasında geçerli girişlerin kontrolü bu bölümde yapılmıştır. Kullanıcının oyunla etkileşimi büyük ölçüde bu modül üzerinden sağlanmaktadır.

5. Çalışma Mantığı

Oyun çalıştırıldığında ilk olarak başlangıç ekranı açılır. Bu ekranda oyunun adı gösterilir ve kullanıcı START butonuna basarak oyuna geçiş yapar. Ardından oyun alanı seçimi ekranı

gelir. Kullanıcı burada yalnızca 4 veya 6 girebilir. Geçersiz bir giriş yapılırsa uyarı mesajı gösterilir ve kullanıcı tekrar doğru seçimi yapmaya yönlendirilir.

Seçim yapıldıktan sonra oyun kartları hazırlanır. setup_cards prosedürü ile kart dizileri sıfırlanır, harfler oluşturulur ve seçilen tablo boyutuna göre gerekli kart sayısı kadar çiftler dizilere yerleştirilir. Daha sonra bu kartlar shuffle_cards ve shuffle_alphabet prosedürleriyle rastgele karıştırılır.

Kartlar hazırlandıktan sonra kısa bir önizleme aşaması gerçekleşir. Bu aşamada tüm kartlar kısa süreliğine açık olarak gösterilir. Bu sayede oyuncu kartların yerlerini hafızasına almaya çalışır. Sonrasında kartlar tekrar kapatılır ve asıl oyun başlar.

Oyuncu, satır ve sütun bilgisi girerek kart seçer. İlk kart açıldıktan sonra ikinci kart seçimi beklenir. Eğer iki kart aynı harfi taşıyorsa eşleşme başarılı olur ve kartlar açık kalır. Yanlış eşleşme durumunda kartlar tekrar kapatılır, hamle sayısı azalır ve süreye ceza uygulanır.

Oyun sırasında kalan hamle sayısı ve süre ekranın üst kısmında gösterilir. Oyuncu tüm çiftleri bulduğunda oyun kazanılır. Hamle sayısı sıfıra düşerse veya süre biterse oyun kaybedilir. Ayrıca restart butonu sayesinde oyun yeniden başlatılabilir ve seçilen tablo boyutu korunarak baştan oynanabilir.

7. Veri Yapıları ve Değişkenler

Projede kullanılan temel veri yapıları kartları, kart durumlarını ve oyun akışını takip etmek için tasarlanmıştır. Assembly dilinde yüksek seviyeli veri yapıları yerine çoğunlukla diziler ve tekil değişkenler kullanılmıştır.

cards dizisi, oyun içindeki kartlarda yer alan harfleri saklamaktadır. Bu dizi, kartların gerçek içeriklerini tutar ve oyun sırasında açık kartlarda hangi harfin görüneceğini belirler.

card_states dizisi, kartların açık mı kapalı mı olduğunu takip etmek için kullanılır. Bu dizideki her eleman, ilgili kartın durumu hakkında bilgi verir. Değer 1 ise kart açık, 0 ise kart kapalıdır.

grid_size değişkeni, seçilen oyun alanını tutar. Kullanıcı 4 veya 6 seçtiğinde bu değer güncellenir ve oyun buna göre şekillenir. Kart sayısı, hamle sayısı ve süre gibi değerler de bu değişkene bağlı olarak ayarlanır.

hamle_sayisi, oyuncunun kaçıncı hamlede olduğunu göstermektedir. Oyun boyunca her tam eşleşme sonrası bu değer artırılır ve ekranda gösterilir.

kalan_hamle, oyuncunun ne kadar hamle hakkı kaldığını saklar. Yanlış eşleşmeler bu değeri azaltır. Hamle sayısı sıfıra düştüğünde oyun kaybedilir.

toplam_cift, seçilen oyun alanına göre bulunması gereken kart çifti sayısını tutar. 4x4 oyunda bu değer 8, 6x6 oyunda 18'dir. Tüm çiftler bulunduğunda oyun kazanılmış olur.

time_left, oyunda kalan süreyi saniye bazında tutar. Bu değer oyun alanına göre başta 120 ya da 240 olarak ayarlanır ve oyun boyunca azalır. Süre sıfıra düştüğünde oyun sona erer.

sel_row ve sel_col, oyuncunun seçtiği kartın satır ve sütun bilgilerini saklar. Bu değişkenler doğrudan giriş işlemlerinde kullanılır.

char_idx, seçilen kartın dizideki indeksini hesaplamak için kullanılır. Satır ve sütun seçimi tamamlandıktan sonra ilgili kartın dizideki konumu bu değişkende tutulur.

first_index, second_index, first_value ve second_value değişkenleri, iki kartlık eşleşme kontrolü sırasında kullanılır. İlk ve ikinci seçilen kartların hem indeksleri hem de içerdikleri harfler bu değişkenlerde saklanır.

restart_requested, restart butonuna basılıp basılmadığını işaret eder. Bu bayrak doğru şekilde algılandığında oyun akışı yeniden başlatma bölümüne yönlendirilir.

restart_prev_down, mouse tıklamasının basılı kalma durumunu takip etmek için kullanılır. Böylece restart butonuna aynı basış için birden fazla tetikleme oluşması engellenir.

game_over, time_up ve combo_streak gibi değişkenler de oyunun genel durumunu yönetir. Bunlar sayesinde oyunun bitiş koşulları, zaman yönetimi ve ardışık doğru eşleşme bonusları kontrol edilir.

8. Kullanıcı Arayüzü

Oyunda kullanıcı arayüzü tamamen metin tabanlı olarak tasarlanmıştır.

Başlangıç ekranı;

Oyun alanı seçimi ekranında kullanıcıya 4x4 ve 6x6 seçenekleri hakkında bilgi verilir. Ayrıca geçersiz giriş yapıldığında uyarı mesajı ve sinyali gönderilerek doğru seçime yönlendirme sağlanır.

```
4x4 icin: 15 hamle, 2.00 sure
6x6 icin: 40 hamle, 4.00 sure
Oyun Alani Seciniz (4 veya 6):
Lutfen gecerli bir alan sayisi giriniz.
```

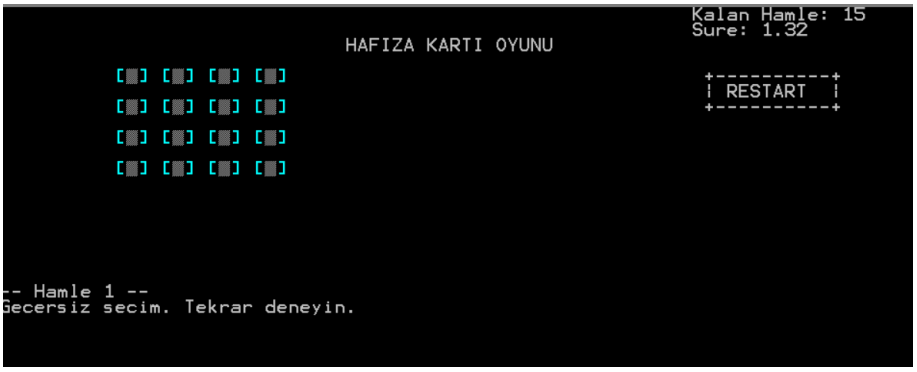
```
4x4 icin: 15 hamle, 2.00 sure
6x6 icin: 40 hamle, 4.00 sure
Oyun Alani Seciniz (4 veya 6): 4
```

Oyundan önceki peek ekranı oyuncunun başlamadan önce kartları kısa süre gözlemlemesini sağlar:

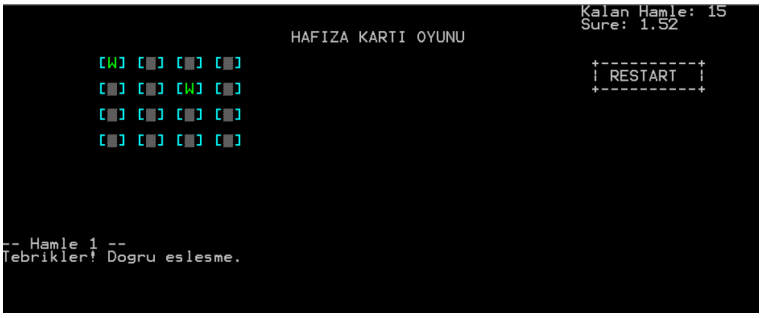
```
                    HAFIZA KARTI OYUNU
[T] [S] [I] [L]                          +----------+
[S] [U] [U] [N]                          | RESTART  |
[E] [N] [M] [I]                          +----------+
[T] [L] [M] [E]
```

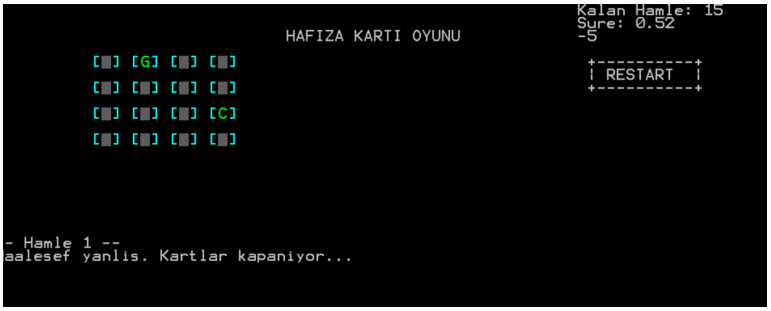
Oyun sırasında ekranın üst kısmında kalan hamle sayısı ve süre bilgisi gösterilir ve kart seçim ekranı belirir. oyun alanının boyutlarından büyük sayılar girilmesi durumunda uyarı mesajı ve sinyali gönderilir:

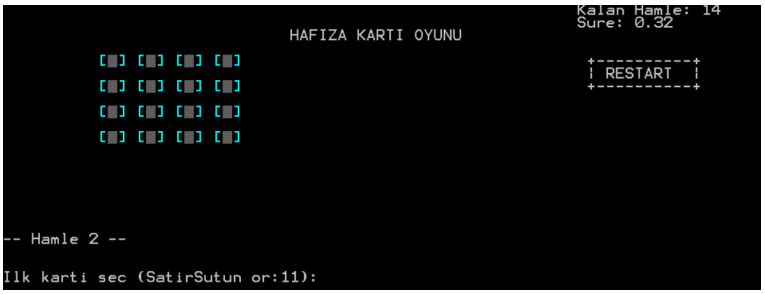
```
                                         Kalan Hamle: 15
                                         Sure: 1.22
                    HAFIZA KARTI OYUNU
[■] [■] [■] [■]                          +----------+
[■] [■] [■] [■]                          | RESTART  |
[■] [■] [■] [■]                          +----------+
[■] [■] [■] [■]

-- Hamle 1 --

Ilk karti sec (SatirSutun or:11):
```



Kartların bulunduğu ana oyun alanı, açık ve kapalı kartları net biçimde ayırt edecek şekilde tasarlanmıştır. Kapalı kartlar blok karakterle gösterilirken, açık kartlar ilgili harflerle görünür. Doğru eşleşme durumunda “Tebrikler! Doğru eşleşme.” mesajı, yanlış eşleşme durumunda ise “Maalesef yanlış. Kartlar kapanıyor...” mesajı gösterilir. Doğru eşlemeden sonraki hamlede kartlar açık kalırken, yanlış eşlemede kartlar kapanır ve süreden 5 saniye düşülür, süre göstergesinin altında ‘-5’ ibaresi belirir:

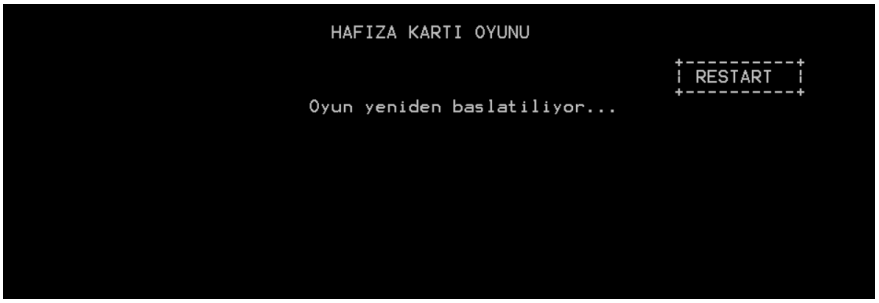




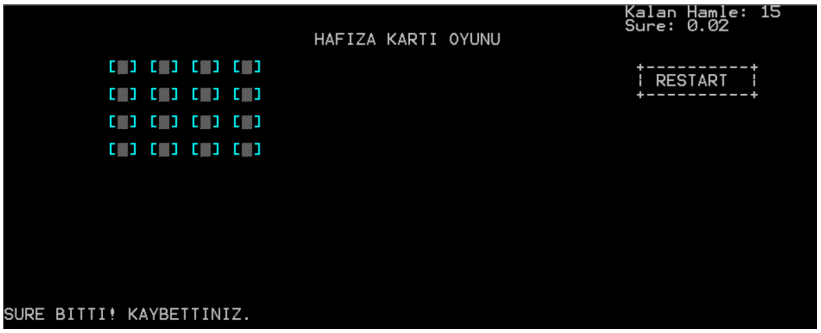


Ard arda doğru tahminler ‘combo’ olarak nitelendirilmektedir. Combo yapılması durumunda süreye 7 saniye eklenir ve Sure göstergesinin altında ‘+7’ ibaresi belirir:

Oyun yeniden başlatılmak istendiğinde ise restart butonuna basılır ve ekranda “Oyun yeniden başlatılıyor..”mesajıı belirir. Sonrasında oyuncunun ilk başta seçtiği oyun alanının boyutlarına uygun olacak şekilde tekrardan kartlar dağıtılır:



Belirtilen sürenin bitmesi, hamle sayının sonlanması veya tüm kartların doğru eşlenmesi durumunda oyun sona erer: (Oyunu kazanamadığım için kazanma mesajının olduğu ekranı koyamadım, aynı şekilde hamle sayısını bitirmeden süreyi bitirdiğimden hamle bitti ekranını koyamadım.)



Not: Oyunun işleyişi 6x6lık için de aynıdır.

9. Kodun Yapısı ve Önemli Fonksiyonların/Prosedürlerin Açıklanması

main.asm dosyasında kodun en başında veri bölümü yer alır: Burada, kullanıcıya döndürülecek mesajlar, tablo boyutları, kart içerikleri ve oyunun kontrolü için flag değerleri tutulmaktadır. Ana akış `main proc` içinde başlar. Bu bölümde veri segmenti DS register’ına yüklenir. Ardından oyun başlangıç ekranı ve seçim ekranı sırasıyla çağrılır.

-show_start_screen Prosedürü

Bu prosedür *display.asm* içinde yer alır ve oyunun başlangıç ekranını oluşturur. Ekranda oyun adı ve START butonu görünür. Kullanıcı butona tıkladığında oyun alanı seçim ekranına geçilir. Kodun temel yapısı şöyle başlar:

```
show_start_screen proc
    call clear_screen
    mov ax, 0000h
    int 33h
    mov ax, 0001h
    int 33h
```

HAFIZA KARTI OYUNU yazısı ekrana basmak için lea dx, msg_title komutu kullanılır.
ve START butonu üç satır halinde çizilir. Butona tıklama kontrolü sol fare tuşu basılana kadar bekler. Tuşa basıldığında koordinatlar kontrol edilir ve tıklamanın buton alanında olup olmadığına bakılır. Böylece kullanıcı yalnızca doğru bölgeye tıkladığında oyuna geçiş yapabilir:

```
wait_click:
    mov ax, 0003h
    int 33h
    test bx, 1
    jz wait_click
```

-read_grid_size Prosedürü

main.asm içinde başlangıç ekranından sonra oyun alanı seçimi yapılır. Kullanıcıdan yalnızca `4` veya `6` girmesi *lea dx, msg_selection* komutu ile belirlenir. Bu bölümde kullanıcıdan klavye ile giriş alınır. Eğer girilen değer `4` veya `6` değilse, geçersiz giriş mesajı gösterilir:

```
cmp al, '4'
    je grid_4x4
    cmp al, '6'
    je grid_6x6

    call play_beep
    lea dx, msg_invalid_grid
    mov ah, 09h
    int 21h
```

-reset_game_state Prosedürü

Bu prosedür oyunun tüm temel sayaçlarını sıfırlamak için kullanılır. Oyun ilk başladığında ya da restart yapıldığında çağrılır. Burada oyun bitiş bayrakları sıfırlanır, combo sayacı sıfırlanır, hamle sayısı 1'den başlatılır ve seçilen tablo boyutuna göre kalan hamle ve süre ayarlanır:

```
reset_game_state proc
    mov [game_over], 0
    mov [time_up], 0
```

```
mov [restart_requested], 0
.
.
.
mov al, [grid_size]
cmp al, 6
jne set_state_4x4

mov [kalan_hamle], 40
mov [toplam_cift], 18
mov [time_left], 240
```

-setup_cards Prosedürü

Kartların oluşturulması ve hazırlanması *logic.asm* içindeki bu prosedürle yapılır. Bu prosedür oyunun en önemli bölümlerinden biridir çünkü kartların içeriğini üretir ve onları oyun tahtasına uygun şekilde hazırlar. İlk olarak sayaçlar ve diziler temizlenir. Sonra `cards` ve `card_states` dizileri sıfırlanır. Kartların içeriği oluşturulmadan önce `alphabet` dizisi karıştırılır, fill loop: ile en son adımda harfler ikişerli olarak yerleştirilir:

```
setup_cards proc
    .
    .
    .
    lea di, cards
    mov cx, 36
    xor al, al
clear_cards_loop:
    mov [di], al
    inc di
    loop clear_cards_loop
    .
    ...
    call shuffle_alphabet
    .
    …
    fill_loop:
    mov al, [si]
    mov [di], al
    inc di
    mov [di], al
    inc di
    inc si
    loop fill_loop
    …
setup_cards endp
```

-draw_grid Prosedürü

`grid.asm` içindeki `draw_grid`, oyun tahtasının ekrana çizilmesini sağlar. Bu prosedür kartların kapalı mı açık mı olduğunu `card_states` dizisinden okuyarak karar verir. Başlangıçta tablo boyutuna göre satır sayısı hazırlanır:

```
draw_grid proc
   mov al, [grid_size]
   mov [tmp_bl], al
   lea si, card_states
```

Eğer kart açık durumdaysa `cards` dizisinden harf alınır:

```
…
show_open_card:
   mov ax, si
   sub ax, offset card_states
   lea di, cards
   add di, ax
   mov al, [di]
```

-reveal_selected_card Prosedürü

Bu prosedür, oyuncunun seçtiği kartı geçici olarak açmak için kullanılır. `input.asm` içinde yer alır. Bu prosedür sayesinde oyuncu bir kartı seçtiğinde o kart geçici olarak görünür hale gelir.

```
reveal_selected_card proc
   mov al, [sel_row]
   mov bl, 2
   mul bl
   add al, 4
   mov dh, al
   ```
   mov ah, 02h
   int 10h

   mov al, [char_idx]
   lea si, cards
   add si, ax
   mov al, [si]
   int 10h
```

**-get_selection Prosedürü**

Bu prosedür, oyuncudan satır ve sütun bilgisini almak için kullanılır. Önce satır girişi beklenir:

```
read_row:
   call read_key_with_timer
   cmp [restart_requested], 1
```
```

```
    je selection_restart
```

Kullanıcının girdiği karakterin geçerli olup olmadığı kontrol edilir. Satır ve sütun sadece 1 ile grid boyutu arasında olmalıdır. Yanlış girişte uyarı verilir:

```
…
    cmp al, '1'
    jb invalid_selection

    mov bl, [grid_size]
    add bl, '0'
    cmp al, bl
    ja invalid_selection
```

```
invalid_selection:
    call play_beep
```

-`eslesme_dogru` ve `eslesme_yanlis` Prosedürleri

Doğru eşleşme durumunda `eslesme_dogru` çalışır. Önce kartların açık duruma geçtiği işaretlenir:

```
mov byte ptr [card_states + si], 1
```

Aynı zamanda açık kart sayısı artırılır ve kalan hamle azaltılır:

```
inc [acik_kart_say]
dec [kalan_hamle]
```

Arka arkaya doğru eşleşme yapılırsa combo bonusu verilir ve süreye ekleme yapılır:

```
inc [combo_streak]
cmp [combo_streak], 2
add word ptr [time_left], 7
```

Yanlış eşleşmede ise kartlar tekrar kapatılır:

```
mov byte ptr [card_states + si], 0
dec [kalan_hamle]
mov [combo_streak], 0
```

Ayrıca süre cezası uygulanır:

```
sub word ptr [time_left], 5
```

- `update_timer` ve `update_time_display`
Zaman yönetimi `display.asm` içinde yer alan `update_timer` ile yapılır. BIOS zaman kesintisi kullanılarak geçen süre takip edilir. Bu yapı yaklaşık olarak saniye bazında güncelleme sağlar. Süre sıfıra ulaşınca `time_up` bayrağı set edilir. Ekrandaki süre göstergesi ise `update_time_display` ile yazdırılır. Burada kalan süre dakika ve saniye biçiminde gösterilir:

```
mov ah, 00h
int 1Ah
mov ax, dx
sub ax, [last_tick]
cmp ax, 18
```

```
mov ax, [time_left]
mov bx, 60
div bx
```

- `restart_game` Fonksiyonu
Restart butonuna basıldığında `restart_game` çalışır. Önce oyun sıfırlanır; sonra ekran temizlenir, yeniden başlatılıyor mesajı gösterilir ve kısa bir bekleme yapılır:

```
restart_game:
   mov byte ptr [restart_requested], 0
   mov byte ptr [restart_prev_down], 0
   call reset_game_state
```

```
   call clear_screen
   call print_title
   call draw_restart_button
   lea dx, msg_restarting
   call restart_pause
```

10. Karşılaşılan Sorunlar ve Çözümler

Proje geliştirme sürecinde Assembly dilinin düşük seviyeli yapısından kaynaklanan çeşitli teknik sorunlarla karşılaşılmıştır. Özellikle ekran çıktıları, kart konumları, kullanıcı girdileri, buton tıklamaları ve zaman yönetimi en çok dikkat gerektiren alanlar olmuştur. Bu sorunlar, kod üzerinde yapılan düzenlemelerle tek tek çözülmüştür.

İlk karşılaşılan sorunlardan biri, kartların ekrana yerleştirilmesi sırasında bazı harflerin yanlış konumda görünmesiydi. Bunun nedeni, kartın satır ve sütun hesaplarının grid çizimiyle tam

uyumlu olmamasıdır. reveal_selected_card prosedüründeki satır hesabı ile draw_grid prosedüründeki başlangıç koordinatları eşleştirilerek bu sorun giderilmiştir.

Bir diğer sorun, başlangıçta kartların diziliminin yeterince rastgele görünmemesiydi. Kart ve harf dizilerinin karıştırılması için ayrı karıştırma prosedürleri kullanılarak daha dengeli bir dağılım sağlanmıştır. Böylece aynı harflerin yan yana veya düzenli bloklar halinde kalması engellenmiştir.

Kullanıcı girişleriyle ilgili de bazı doğrulama sorunları yaşanmıştır. Oyun alanı seçiminde 4 ve 6 dışında geçersiz değerler girilebiliyordu. Bu nedenle seçim ekranına uyarı mesajı eklenmiş ve kullanıcı doğru giriş yapana kadar tekrar seçim yapmaya yönlendirilmiştir. Aynı şekilde kart seçiminde geçersiz satır ve sütun girişleri de kontrol altına alınmıştır.

Kartlar açıldıktan sonra yanlış eşleşme durumunda bazı kartların açık kalması ya da ekranın karışık görünmesi gibi problemler oluşmuştur. Bu durum, kart durum dizisinin doğru şekilde güncellenmesi ve oyun tahtasının yeniden çizdirilmesi ile çözülmüştür. card_states dizisi sayesinde hangi kartların açık veya kapalı olduğu güvenilir biçimde takip edilmiştir.

Restart butonunun çalışması da geliştirme sürecinde ayrı bir sorun alanı olmuştur. Butonun bazen yalnızca belirli anlarda tepki vermesi, bazen de yanlış tetiklenmesi gibi durumlar ortaya çıkmıştır. Bu sorun, mouse tıklamasının daha dikkatli kontrol edilmesi ve restart bayrağının yalnızca gerçek tıklamada set edilmesi ile çözülmüştür.

Zaman ve hamle göstergeleri de başlangıçta bazı kararsızlıklar göstermiştir. Süre kontrolünün yanlış yerlerde yapılması ya da fazla sık güncellenmesi, sürenin erken bitmiş gibi görünmesine neden olmuştur. Zaman güncelleme işlemleri tek bir akışta düzenlenerek bu problem azaltılmıştır. Böylece oyuncunun süre takibi daha doğru ve güvenilir hale getirilmiştir.

11. Gelecek Geliştirmeler

Bu proje mevcut haliyle temel işlevlerini yerine getirmektedir; ancak gelecekte daha da geliştirilebilir. En önemli geliştirme alanlarından biri, farklı zorluk seviyelerinin eklenmesidir. 4x4 ve 6x6 dışında daha büyük ya da daha küçük tablolar eklenerek oyun çeşitlendirilebilir.

Bir diğer geliştirme alanı, puanlama sistemidir. Hamle sayısı, kalan süre, ardışık doğru eşleşmeler gibi kriterler dikkate alınarak daha ayrıntılı bir skor sistemi oluşturulabilir. Bu sayede oyuncular kendi performanslarını daha net değerlendirebilir.

Oyunun menü yapısı da genişletilebilir. Yardım ekranı, yeniden başlatma seçenekleri, yüksek skor kaydı ve ayarlar ekranı eklenerek oyun daha kapsamlı hale getirilebilir. Bu tür eklemeler, kullanıcı deneyimini zenginleştirir ve oyunu daha uzun süre oynanabilir kılar.

Görsel ve ses tarafında da geliştirmeler yapılabilir. Daha düzenli bir arayüz, renkli kart görünümü, daha belirgin ses efektleri ve doğru/yanlış durumları için farklı geri bildirimler oyunu daha çekici hale getirebilir. Böylece proje yalnızca teknik açıdan değil, kullanıcı deneyimi açısından da daha güçlü bir yapıya ulaşabilir.

12. Github Linki

https://github.com/sumeyyekarakosee/8086-Memory-Card-Game